

VIB-Leuven Mini-Hackathon Outputs

Presenters: Buğra Özdemir, Benjamin Pavie



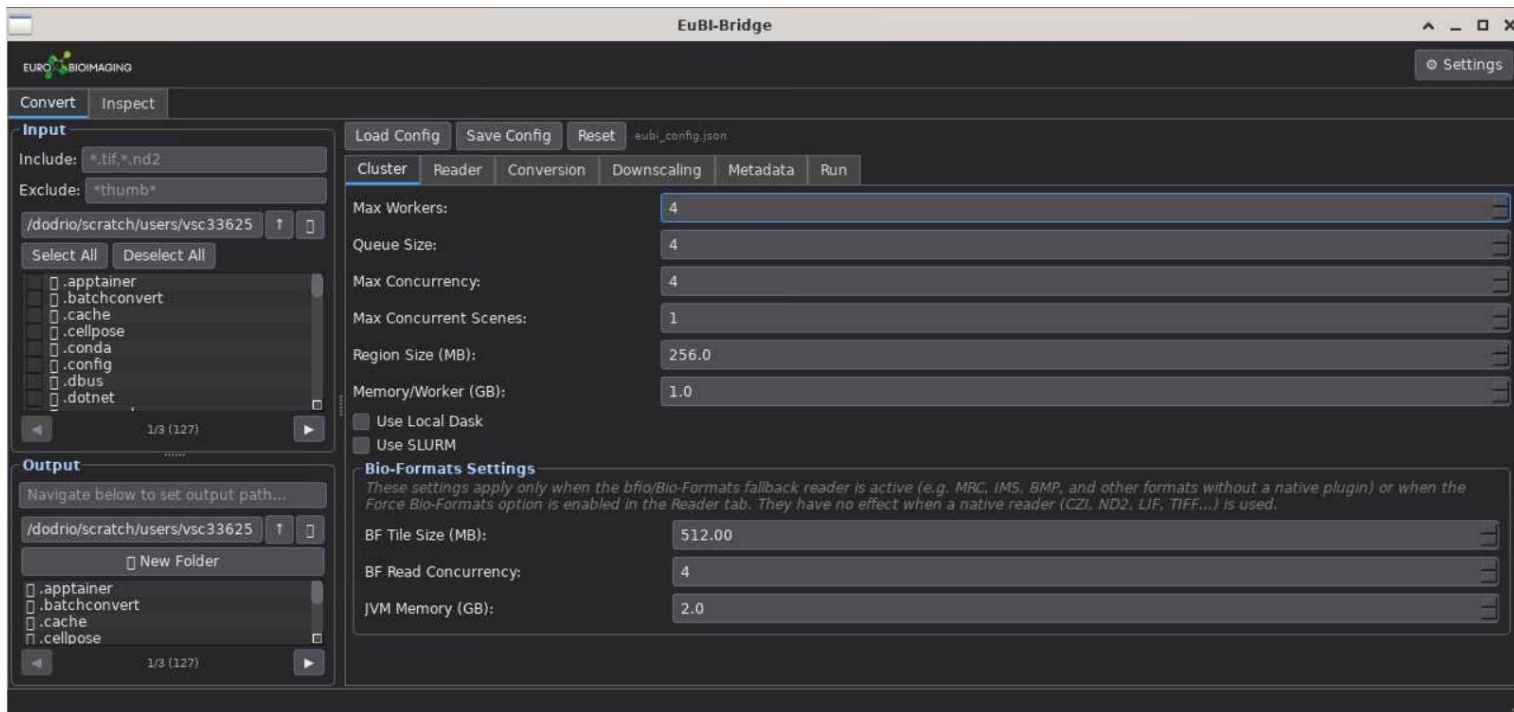
05.06.2026

What is EuBI-Bridge

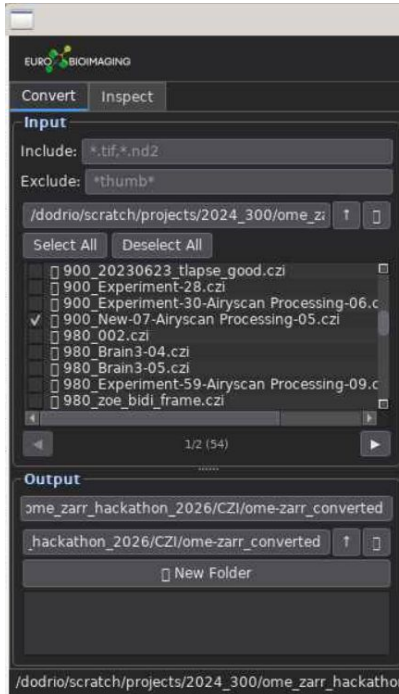
An intuitive tool for data conversion, inspectio microscopy images to the OME-Zarr format.

- **Batch conversion** — convert large collections on cluster or local workstation
- **Two conversion modes** — Unary vs Aggregative
- **Wide format support** — reads CZI, ND2, LIF, TIFF and more via native readers or Bio-Formats
- **Pyramidal downscaling** — auto-generates multi-resolution layers
- **Integrated viewer & metadata editor** — inspect and correct pixel sizes and channel metadata in-place
- **HPC-ready** — runs via conda, Docker or Singularity; Nextflow CLI module available

How the GUI looks like

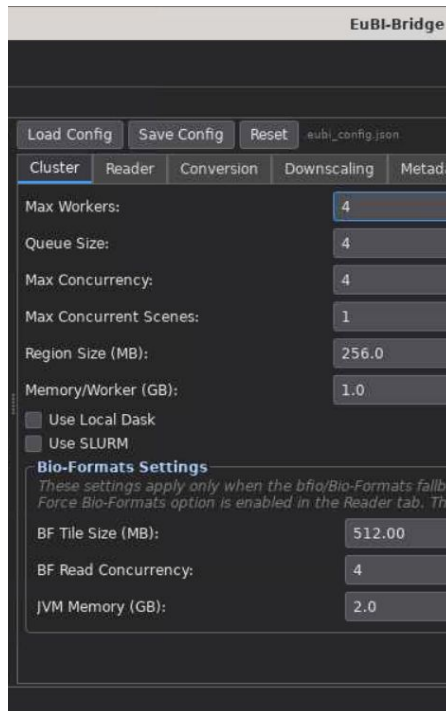


Specify the input and output



1. Choose the input and output directories in the browser in the left sidebar
2. Select the actual images for batch-conversion

Specify the compute resources



The screenshot shows the EuBI-Bridge configuration interface. At the top, there are buttons for 'Load Config', 'Save Config', and 'Reset', along with a file name 'eubi_config.json'. Below these are tabs for 'Cluster', 'Reader', 'Conversion', 'Downscaling', and 'Metadata'. The 'Cluster' tab is selected. It contains several input fields for resource configuration: 'Max Workers' (4), 'Queue Size' (4), 'Max Concurrency' (4), 'Max Concurrent Scenes' (1), 'Region Size (MB)' (256.0), and 'Memory/Worker (GB)' (1.0). There are also checkboxes for 'Use Local Dask' and 'Use SLURM'. A section titled 'Bio-Formats Settings' is expanded, showing 'BF Tile Size (MB)' (512.00), 'BF Read Concurrency' (4), and 'JVM Memory (GB)' (2.0). A note below the Bio-Formats settings states: 'These settings apply only when the bfio/Bio-Formats fallback is used. Force Bio-Formats option is enabled in the Reader tab. The'.

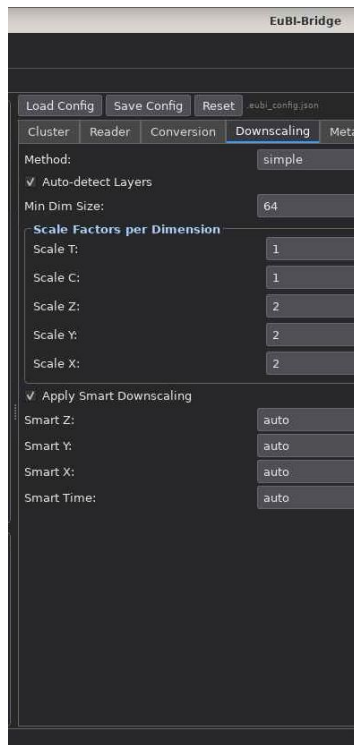
1. Click the cluster tab
2. Specify the resources. Some important parameters:
 - a. Max Workers: a single worker creates a single OME-Zarr container
 - b. Max Concurrency: number of threads per worker
 - c. Region Size (MB): memory allocated for each written region

Specify the conversion parameters

The screenshot shows the 'eURL Config' application interface. At the top, there are buttons for 'Load Config', 'Save Config', and 'Reset', followed by the filename 'eurl_config.json'. Below these are tabs for 'Cluster', 'Reader', 'Conversion' (which is selected), 'Downscaling', 'Metadata', and 'Run'. The 'Conversion' tab contains several sections: 'Data Type' with a dropdown set to 'auto'; a list of checkboxes for 'Verbose', 'Overwrite', 'Squeeze Dimensions' (checked), 'Save OME-XML' (checked), and 'Skip Dask'; 'Compression' settings including 'Codec' (blosc), 'Level' (5), and 'Blosc options' (Inner codec: lz4, Shuffle: shuffle); 'Chunking' settings with 'Auto Chunk' checked and 'Target Chunk MB' set to 1.00; 'Zarr Format and Sharding' with 'Format version' set to v2; 'Dimension Ranges (start,stop)' for T, C, Z, Y, and X axes, all set to 0,100; and 'Concatenation' with a 'Time tag' set to 'e.g., t for time tag'.

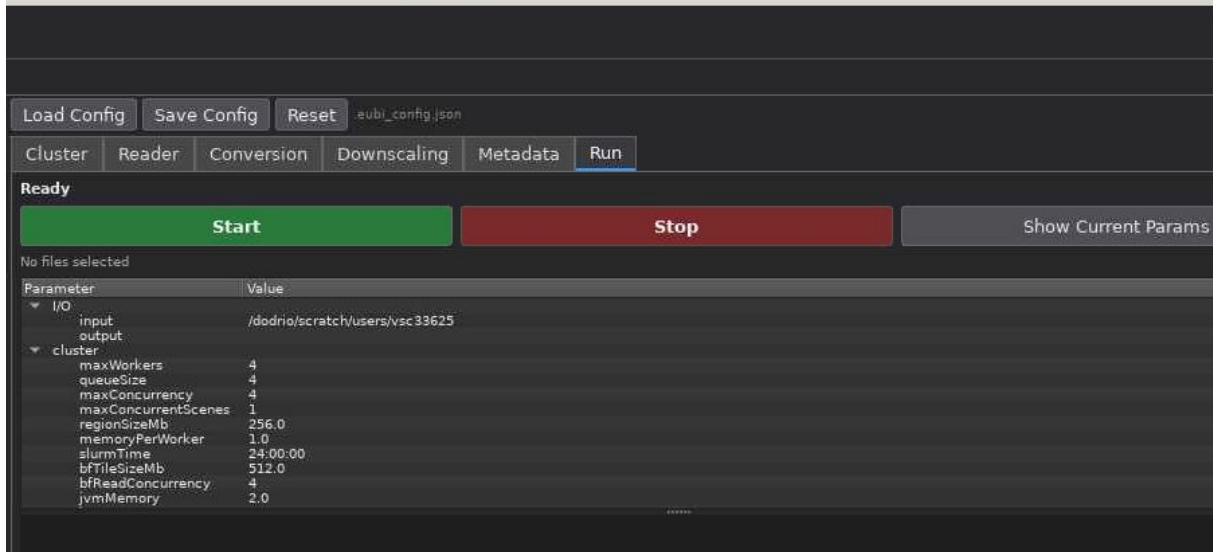
1. Click the Conversion tab.
2. Important parameters:
 - a. Compression
 - i. Auto-chunk
 - ii. Manually chunk
 - b. Chunking
 - c. Zarr format and Sharding
 - d. Concatenation

Step 5: Specify the downsampling



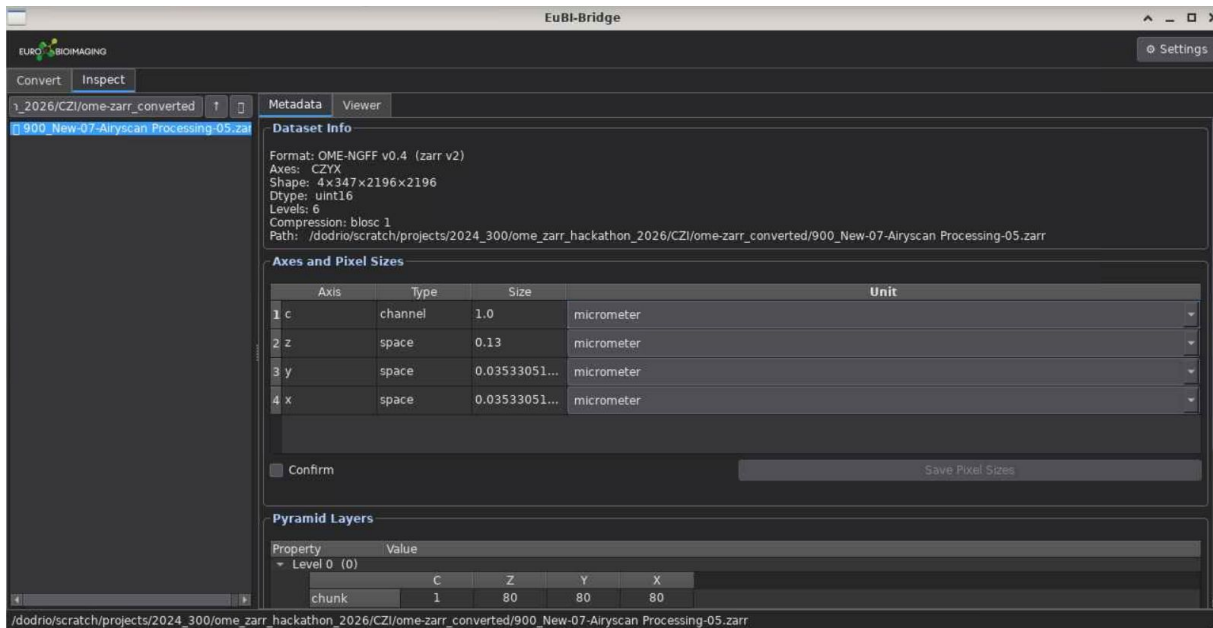
1. Click the Downsampling tab.
2. Important parameters:
 - a. Method: one of different window-based downsampling methods
 - b. Auto-detect Layers: downscale until “Min Dim Size” is reached
 - i. Or the number of layers can be manually specified
 - c. Scale Factor per Dimension: specify scale factor per dimension.
 - i. Or perform isotropic downsampling by ticking “Apply smart-downsampling”

Step 6: Run the workflow



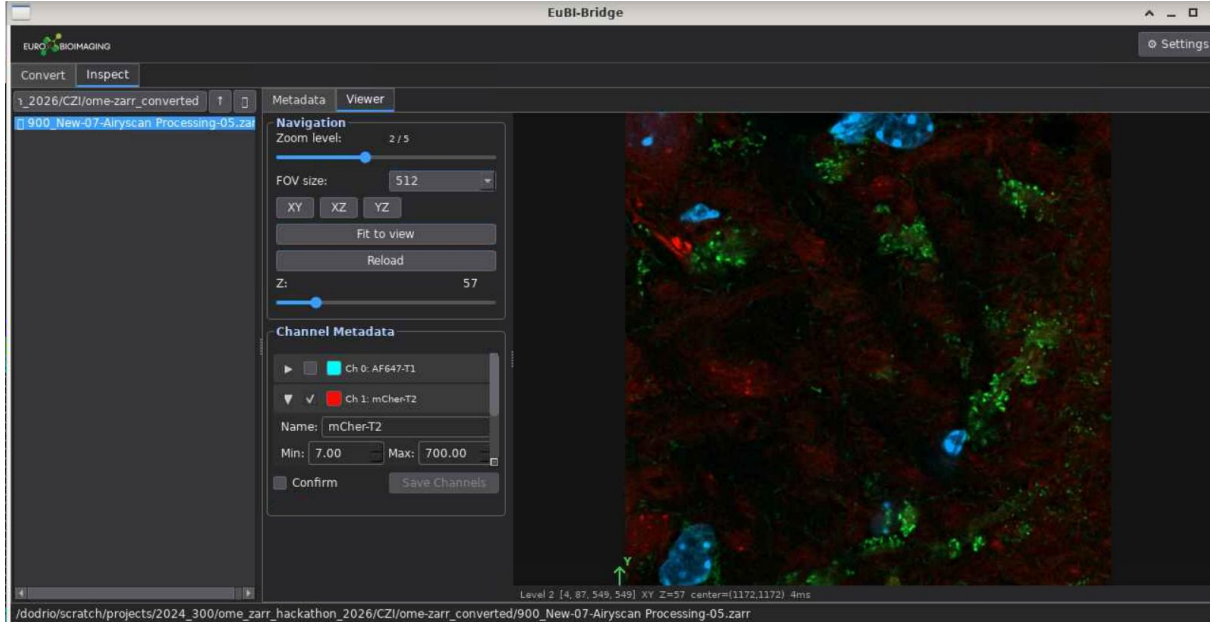
1. Review the settings specified
2. Hit the “Start” button to run the workflow

Step 7: Inspect/Edit the Output Metadata



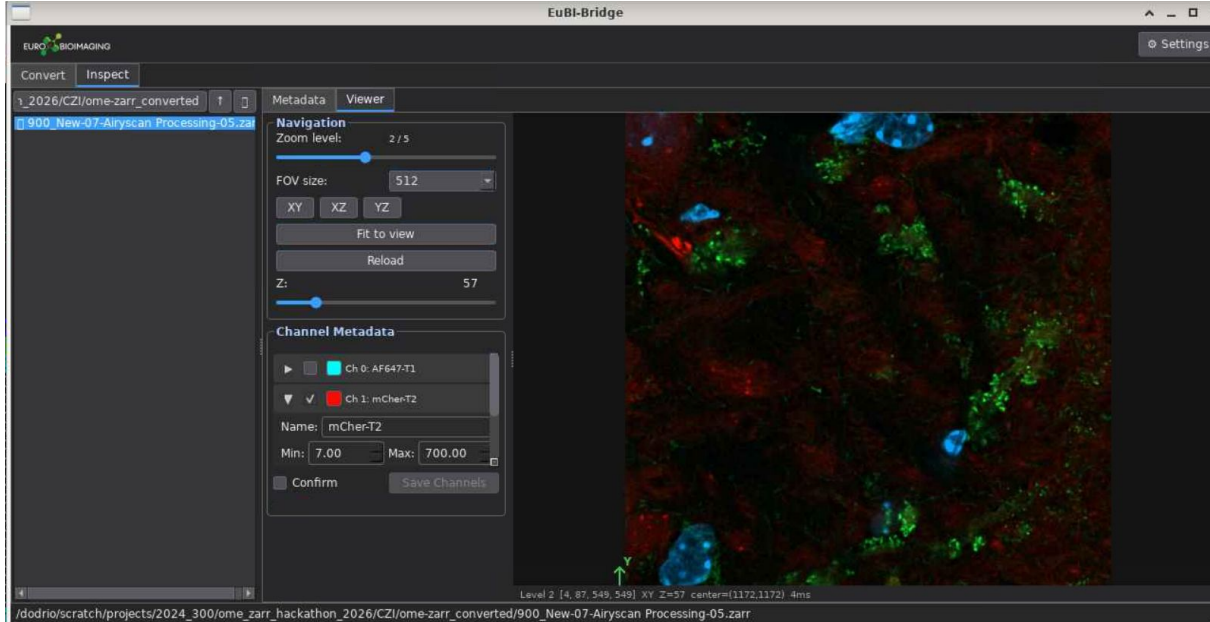
1. Go to the “Inspect/Metadata” tab.
2. Check the metadata under “Axes and Pixel Sizes”
3. Correct the wrong metadata
4. Tick “Confirm”
5. Hit “Save Pixel Sizes”

Step 8: View the Output Data



1. Go to the “Inspect/Viewer” tab.
2. Use the left sidebar to browse to the directory with OME-Zarr. Single click on the OME-Zarr data to open it in the viewer
3. Pan using left mouse button. Scroll through z slices using mouse scroll
4. Optimise the color and intensity range under “Channel Metadata”

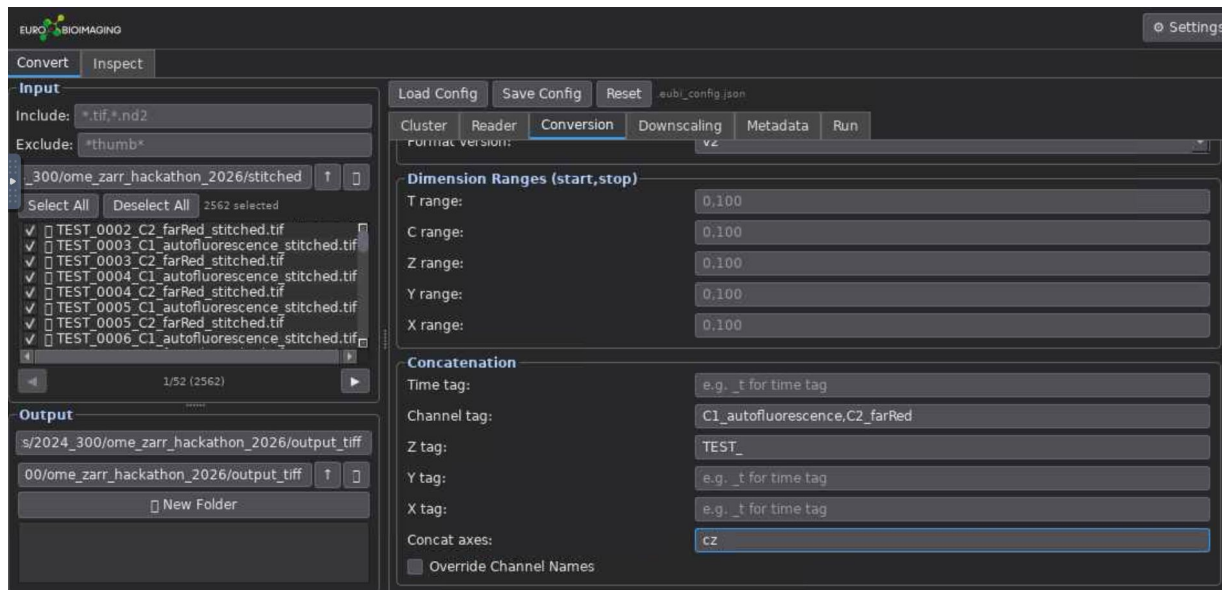
Step 9: Edit the Channel Metadata



After optimising visualisation metadata:

tick “Confirm” and hit
“Save Channels” to edit
the “omero metadata” in
place

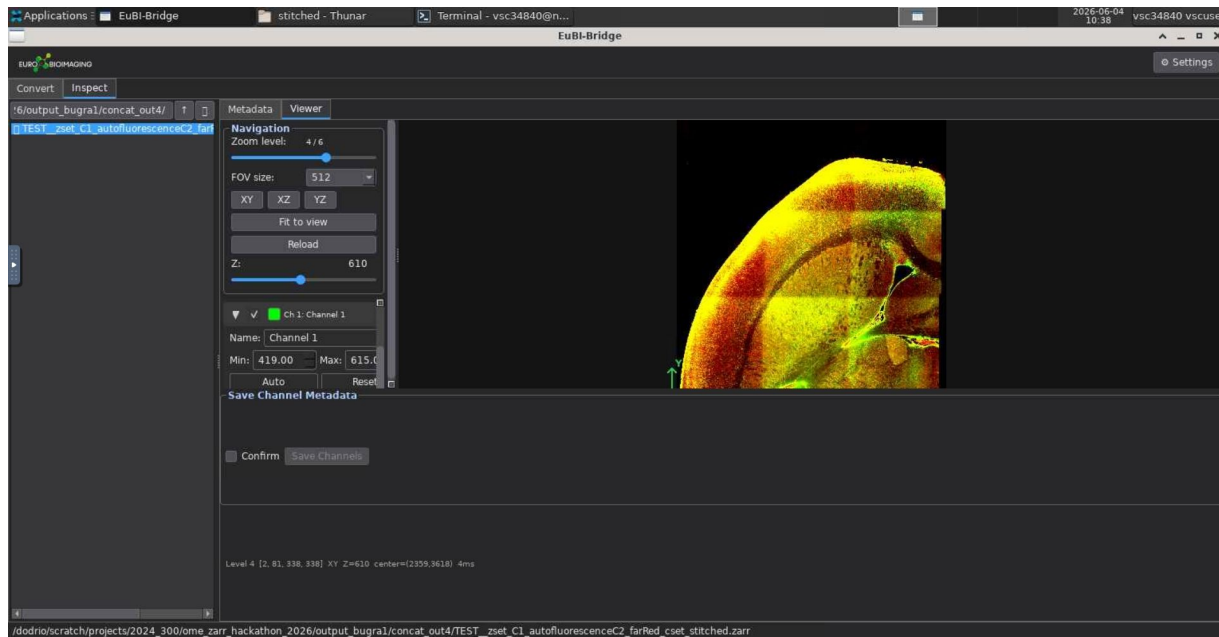
Specific Case: Aggregative Conversion



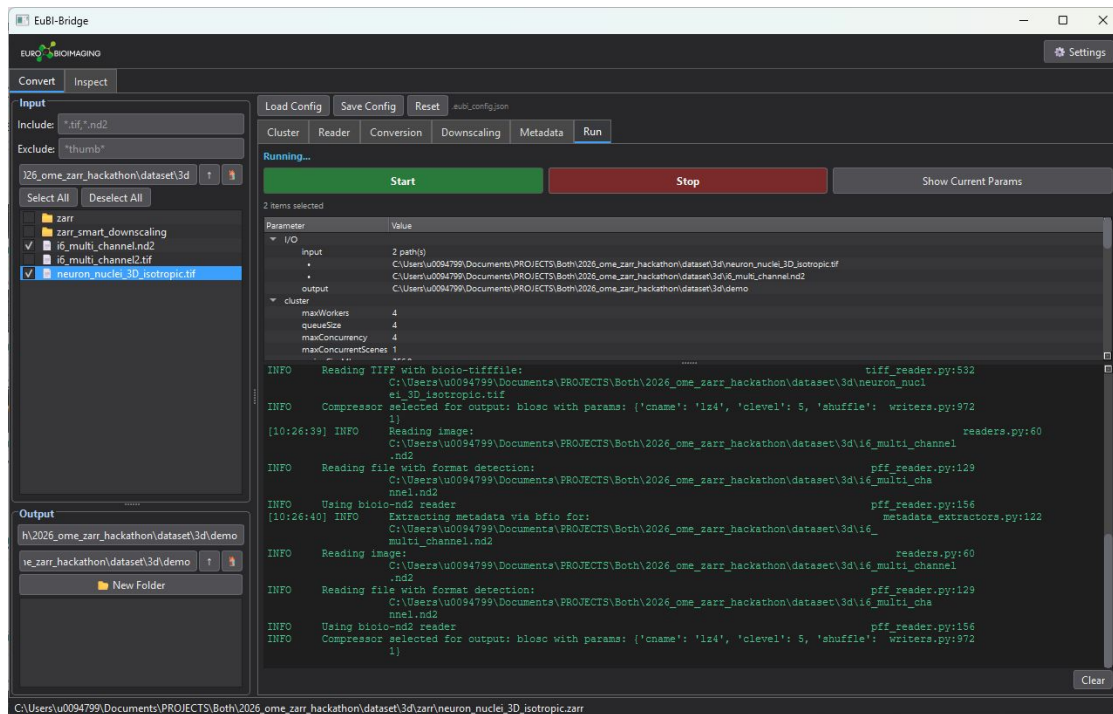
Concatenation requires dimension details to be provided.

Under the “Conversion” tab, use the “Concatenation” section to provide tags per dimension to inform the tool about how to group the files together

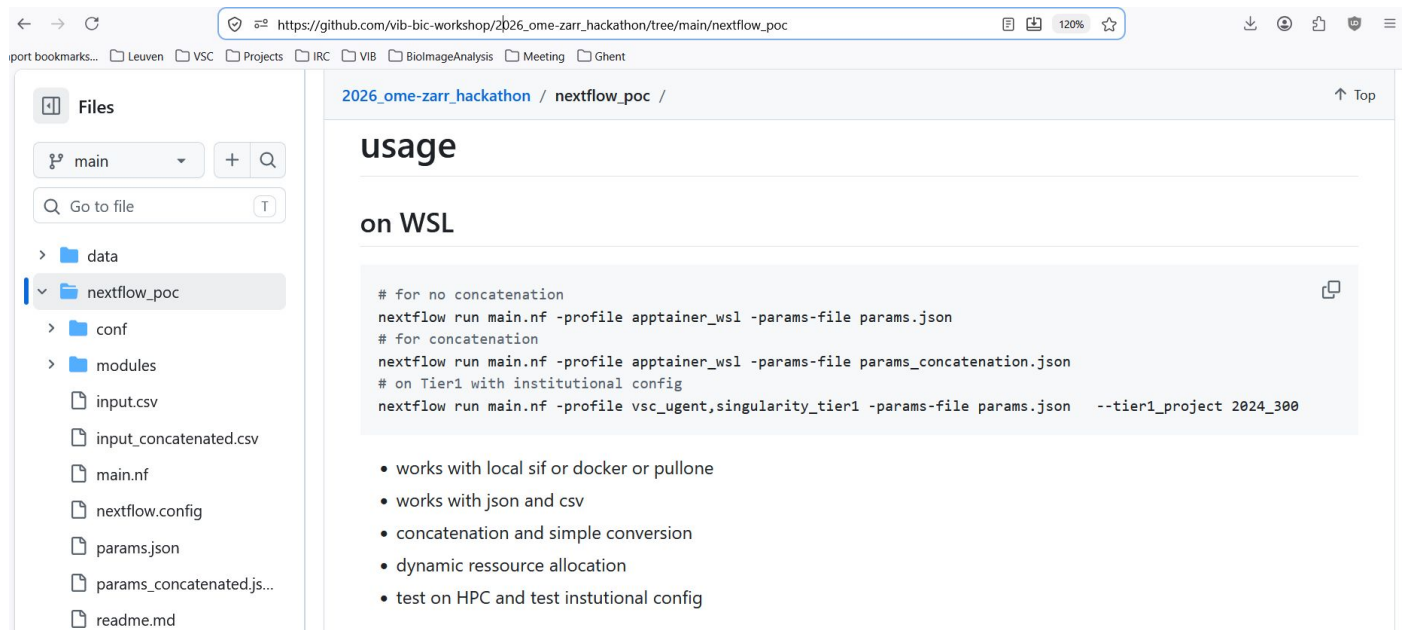
Specific Case: Concatenation



Demo



Nextflow module



The screenshot shows a web browser displaying the GitHub repository page for the 'nextflow_poc' module. The browser's address bar shows the URL: https://github.com/vib-bic-workshop/2026_ome-zarr_hackathon/tree/main/nextflow_poc. The page title is '2026_ome-zarr_hackathon / nextflow_poc /'. The left sidebar shows the file structure of the repository, with the 'nextflow_poc' folder selected. The main content area displays the 'usage' section, which includes a heading 'usage' and a sub-heading 'on WSL'. Below the sub-heading, there is a code block containing the following text:

```
# for no concatenation
nextflow run main.nf -profile apptainer_wsl -params-file params.json
# for concatenation
nextflow run main.nf -profile apptainer_wsl -params-file params_concatenation.json
# on Tier1 with institutional config
nextflow run main.nf -profile vsc_uagent,singularity_tier1 -params-file params.json --tier1_project 2024_300
```

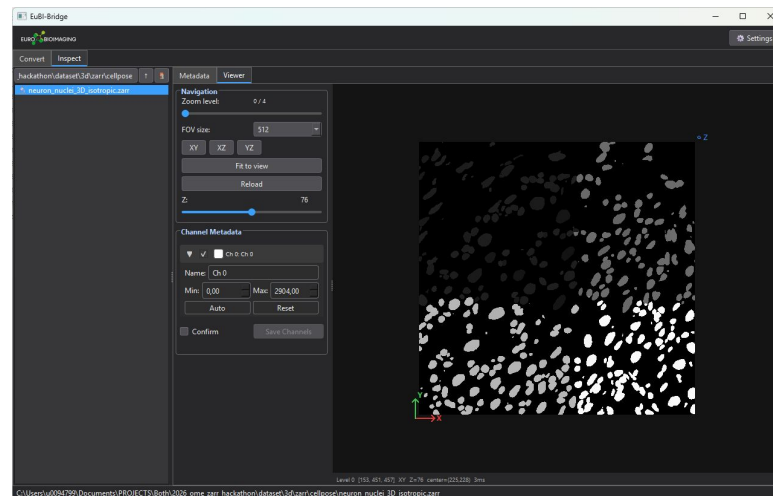
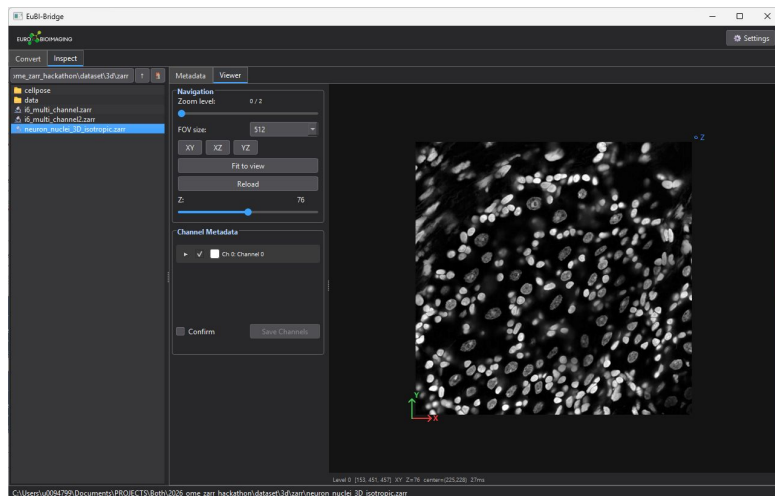
Below the code block, there is a list of bullet points:

- works with local sif or docker or pullone
- works with json and csv
- concatenation and simple conversion
- dynamic ressource allocation
- test on HPC and test insttutional config

https://github.com/vib-bic-workshop/2026_ome-zarr_hackathon/tree/main/nextflow_poc

Cellpose module

Jupyter notebook to run cellpose distributed on ome.zarr



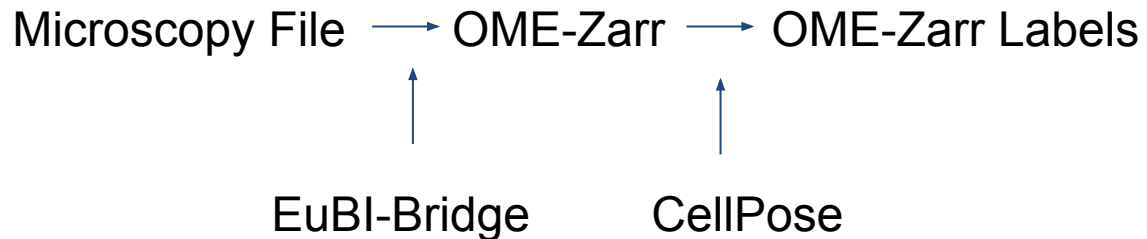
https://github.com/vib-bic-workshop/2026_ome-zarr_hackathon/blob/main/notebooks/cellpose_ome_zarr.ipynb

What we achieved during the hackathon

- Installed EuBI-Bridge in the VIB-Leuven cluster and a local workstation in a conda environment
- Built a docker image of EuBI-Bridge
- Pulled the image into the cluster, converted into a singularity image and launched the GUI from it
- Performed various conversions on the data from VIB
- Updated the tool to improve its robustness in the cluster conditions
- Developed a Nextflow module for EuBI-Bridge's CLI

Outlook

Create a demo nextflow workflow



Develop a binary executable